

# 파이썬 시작하기

## (개발 환경 구축)

# 시작하기 전에

---

- 프로그래밍 (programming)
  - 프로그램을 만드는 것
- 프로그램 (program)
  - 미리 작성된 것

Pro + Gram = ProGram  
미리 + 작성된 것 = 미리 작성된 것

프로그램 = 미리 작성된 것 = 진행 계획

# 시작하기 전에

- 컴퓨터 프로그램 (computer program)
  - 컴퓨터가 무엇을 해야 할지 미리 작성한 진행 계획



# 시작하기 전에

---

- 이진 숫자 (binary digit)
  - 0과 1로 이루어진 컴퓨터의 언어
  - 이진 코드 (binary code) : 이진 숫자로 이루어진 코드
- 프로그래밍 언어 (programming language)
  - 사람이 이해하기 쉬운 언어로 프로그램을 만들기 위해 만들어짐
- 소스 코드 (source code)
  - 프로그래밍 언어로 작성한 프로그램
- 코드 실행기
  - 프로그래밍 언어는 컴퓨터가 이해할 수 없으므로 이를 이진 숫자로 변환해 주는 역할

# 시작하기 전에

---

- 파이썬 (Python)

- 1991년 귀도 반 로섬 (Guido van Rossum)이 개발
- 초보자가 쉽게 배울 수 있는 프로그래밍 언어



- 파이썬의 장점

- 비전공자도 쉽게 배울 수 있음
- 다양한 분야에서 활용할 수 있음
- 대부분의 운영체제에서 동일하게 사용됨

- 파이썬의 단점

- C언어에 비해 일반적으로 10~350배 느림
- 최근에는 컴퓨터 성능이 좋아져 연산이 많이 필요한 프로그램이 아니라면 차이를 크게 느낄 수 없음

# 시작하기 전에

- 개발 환경

- 프로그래밍을 할 수 있는 환경

- 텍스트 에디터

- 프로그래밍 언어로 이루어진 코드를 작성
- 코드 실행기를 이용하여 텍스트 에디터가 작성한 코드를 실행

- 텍스트 에디터

- 파이썬 코드를 입력

- 파이썬 인터프리터

- 파이썬 코드를 실행

```
require "rubygems"
string = "paraiso"
symbol = :paraiso
fixnum = 0
float = 0.00
array = Array.new
array = ["rubens@!@"]
hash = {"test" => "test"}
regexp = /[abc]/

# This is a comment
class Person
```

텍스트 에디터  
: 코드를 작성합니다.

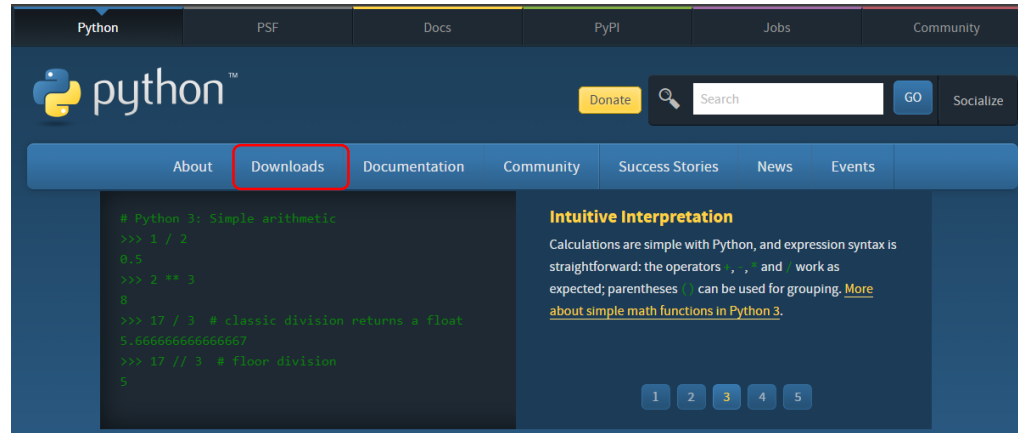


코드 실행기  
: 코드를 실행합니다.

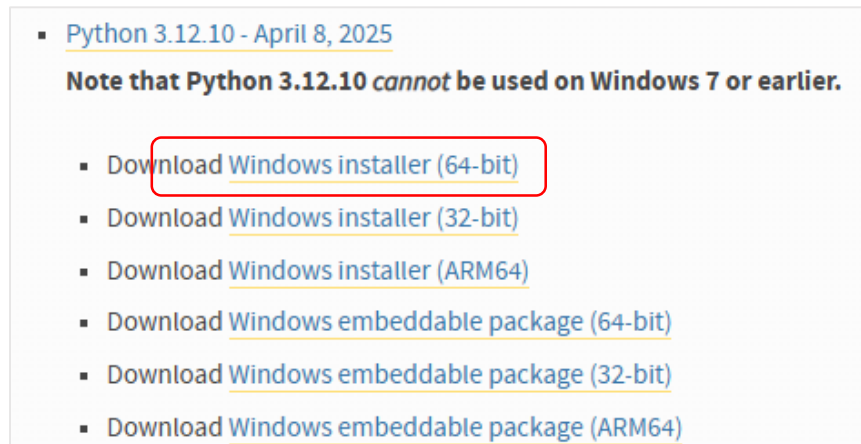
# 개발 환경 설정하기

## ● (실습) 파이썬 설치하기

1. 파이썬 공식 웹 사이트(<https://www.python.org>) 접속하고 [Downloads] 버튼 클릭



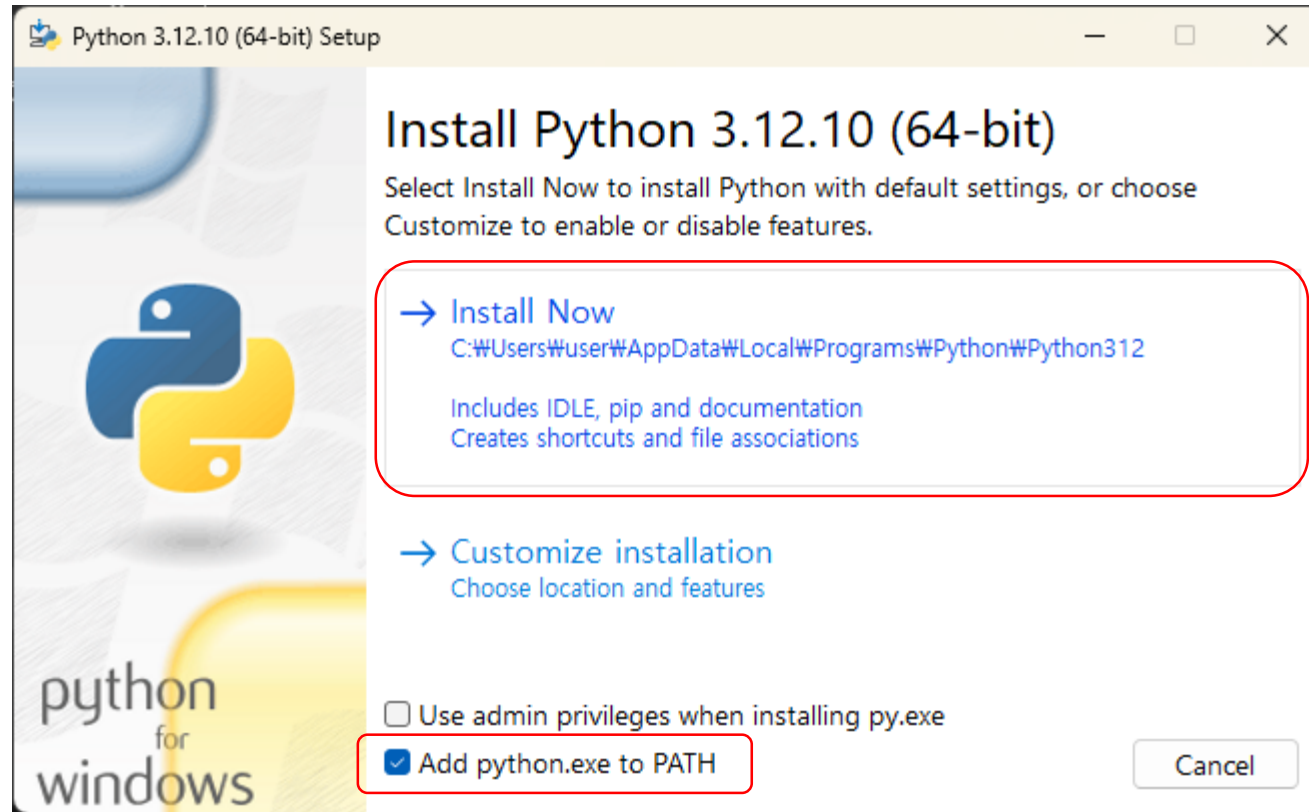
2. Release version에서 [Python 3.12.10] 버전을 찾아 [Download]를 클릭



# 개발 환경 설정하기

- (실습) 파이썬 설치하기

3. 설치 파일 실행 후 [Add python.exe to PATH] 앞을 클릭해 체크 표시한 뒤 [Install Now] 클릭

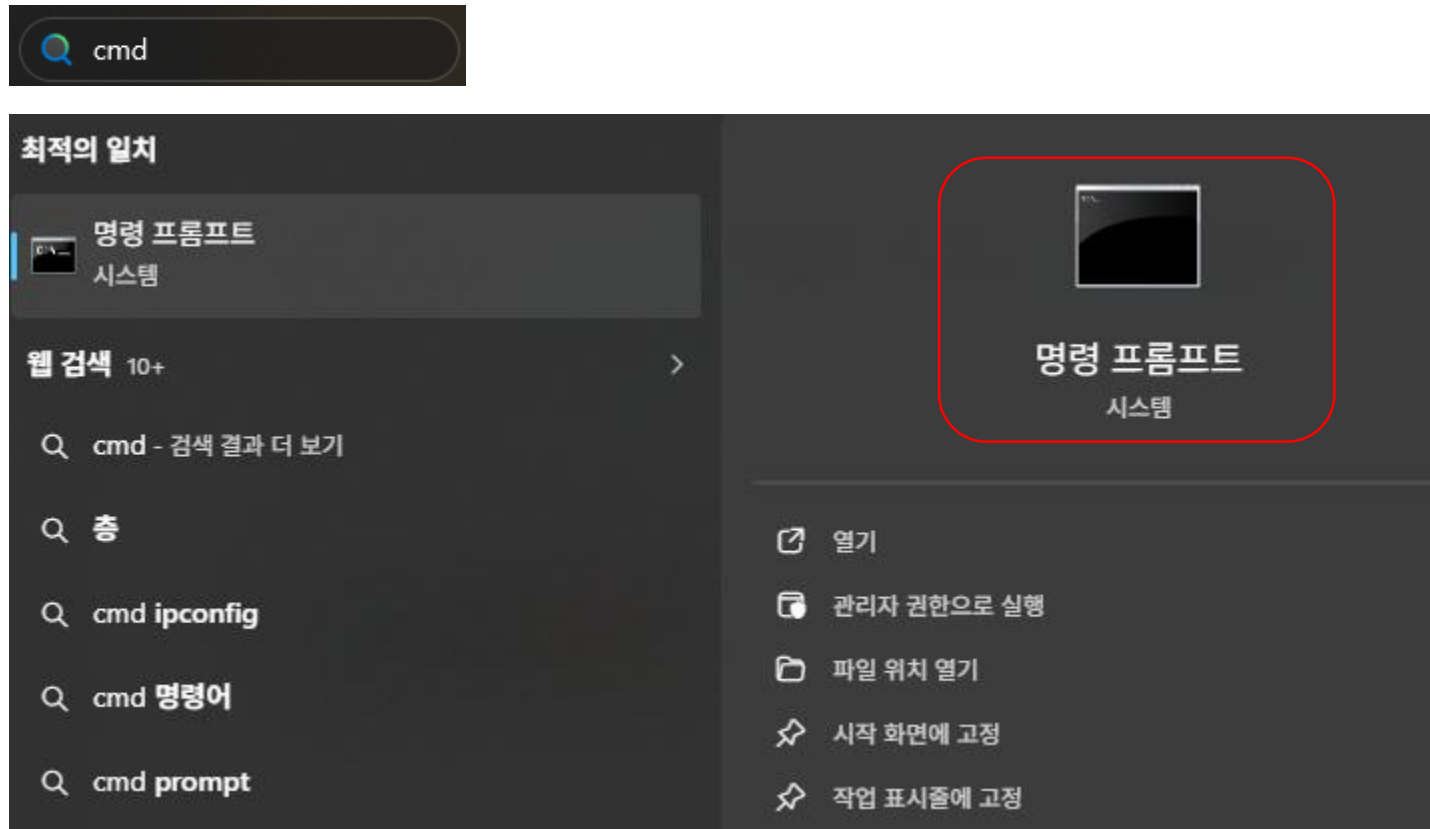




# 개발 환경 설정하기

- (실습) 파이썬 설치하기

4. 윈도우 검색 창에 'cmd'를 입력하고 [명령 프롬프트] 클릭

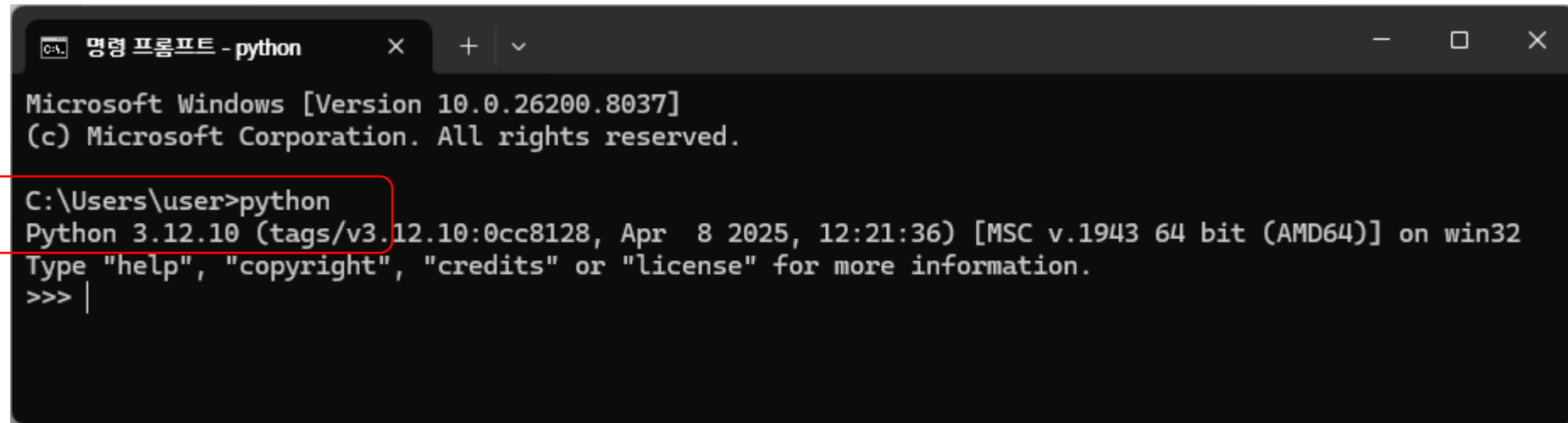


# 개발 환경 설정하기

- (실습) 파이썬 설치하기

5. 명령 프롬프트에서 'python'을 입력해 파이썬의 3.12.10 버전이 잘 설치된 것을 확인

- 인터프리터 (interpreter)
  - 파이썬으로 작성된 코드를 실행해주는 프로그램
- 파이썬 인터랙티브 셸
  - 파이썬 명령어를 한 줄씩 입력하며 실행결과 볼 수 있는 공간



```
명령 프롬프트 - python
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>python
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> |
```

# 개발 환경 설정하기

- (실습) 파이썬 설치하기

5. 명령 프롬프트에서 'python'을 입력해 파이썬의 3.12.10 버전이 잘 설치된 것을 확인

- 프롬프트 (prompt)
  - >>>
  - 코드를 한 줄씩 입력
  - 인터랙티브 셸 = 대화형 셸
    - 컴퓨터와 상호 작용하는 공간이며, 한 마디씩 주고받는 것처럼 대화한다고 하여 대화형 셸로 불리기도 함

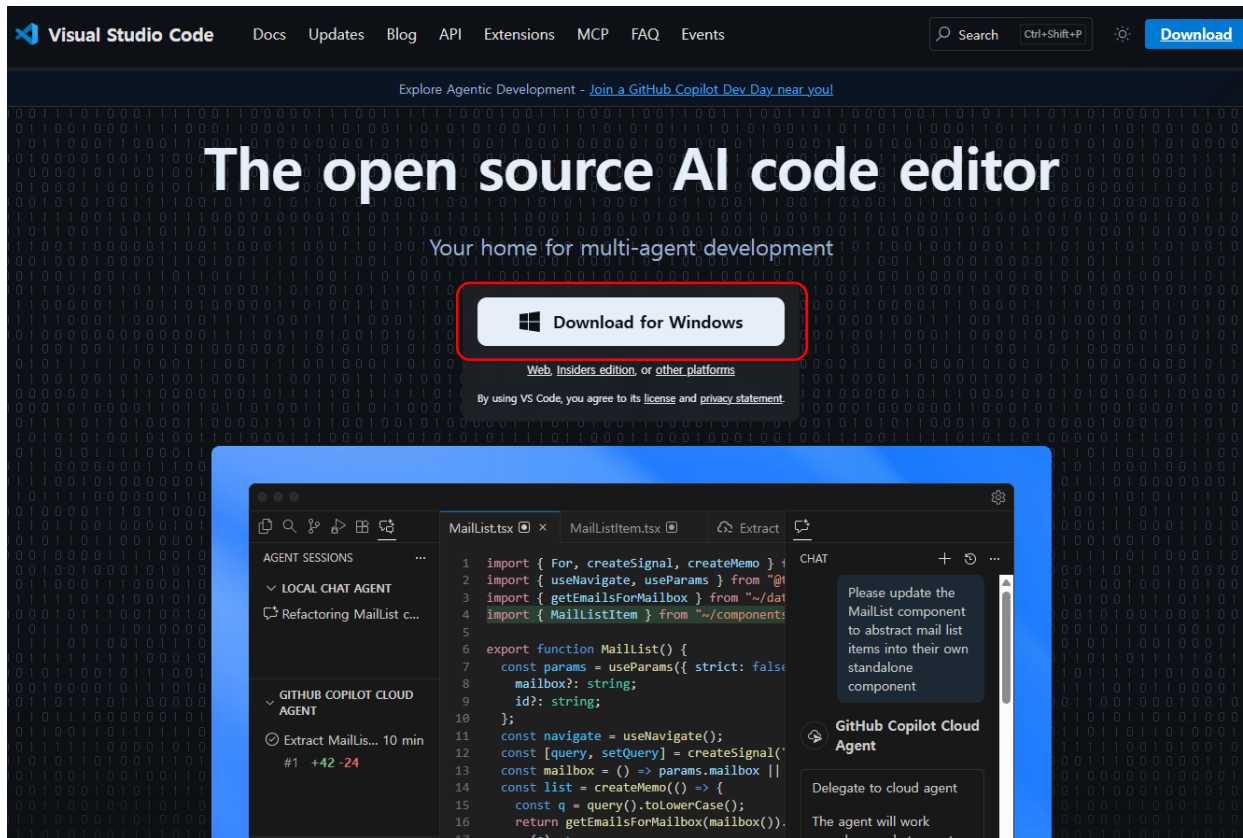
```
>>> 10 + 10 Enter → 10 + 10을 입력하니
20 → 10과 10을 더해 20을 출력합니다.
>>> "Hello" * 3 Enter → Hello라는 문자열을 3번 출력하라는 의미이며,
'HelloHelloHello' → 'HelloHelloHello'를 출력합니다.
>>>
```

# 개발 환경 설정하기

- (실습) 비주얼 스튜디오 코드 설치하기

1. 설치 및 실행

- <https://code.visualstudio.com>



- Visual Studio Code 기본 설정

- 코딩 글꼴 설치

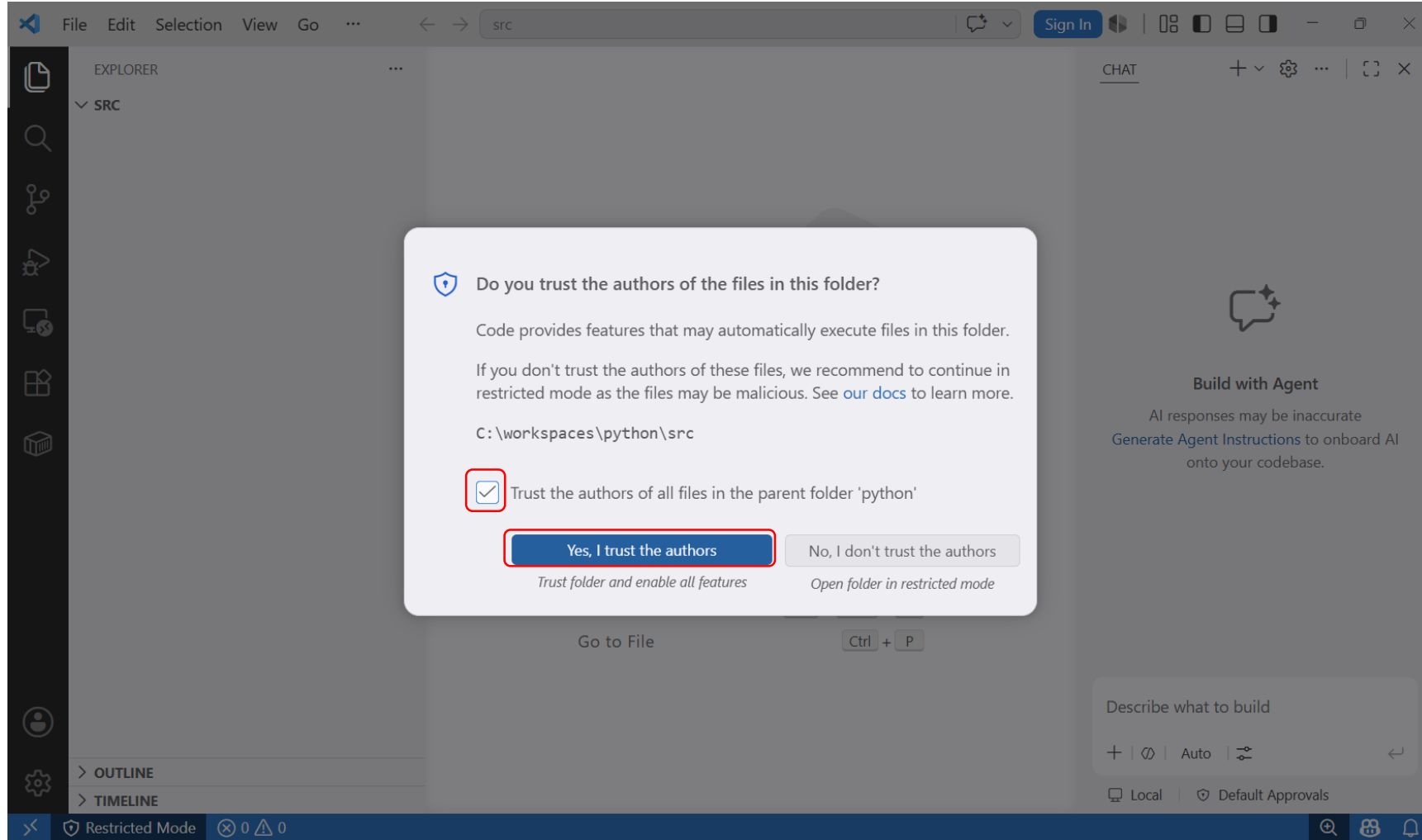
- 한글까지 지원하는 D2 Coding 글꼴 추천

- <https://github.com/naver/d2codingfont> 에서 다운로드한 후 글꼴 설치

# 개발 환경 설정하기

- (실습) 비주얼 스튜디오 코드 설치하기

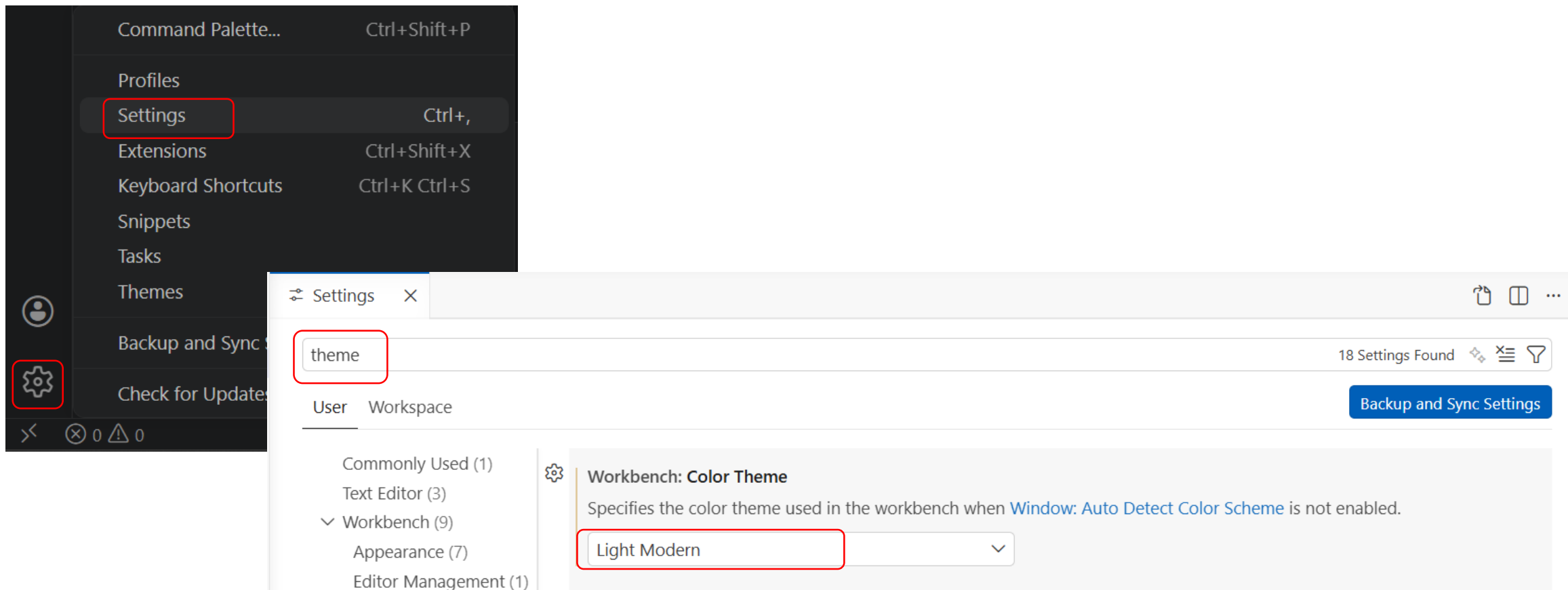
1. 설치 및 실행



# 개발 환경 설정하기

- (실습) 비주얼 스튜디오 코드 설치하기

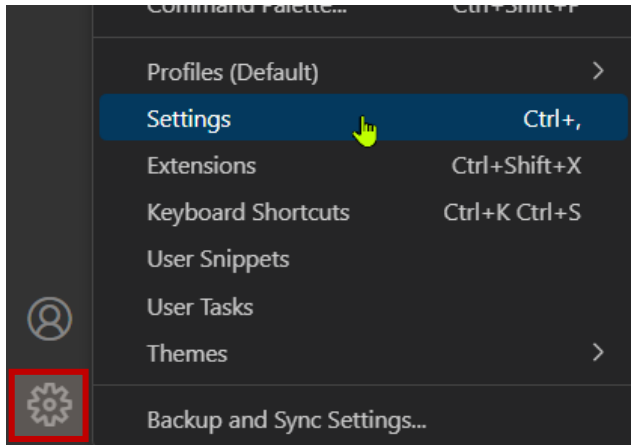
## 2. VS Code 환경 설정과 도구 설치



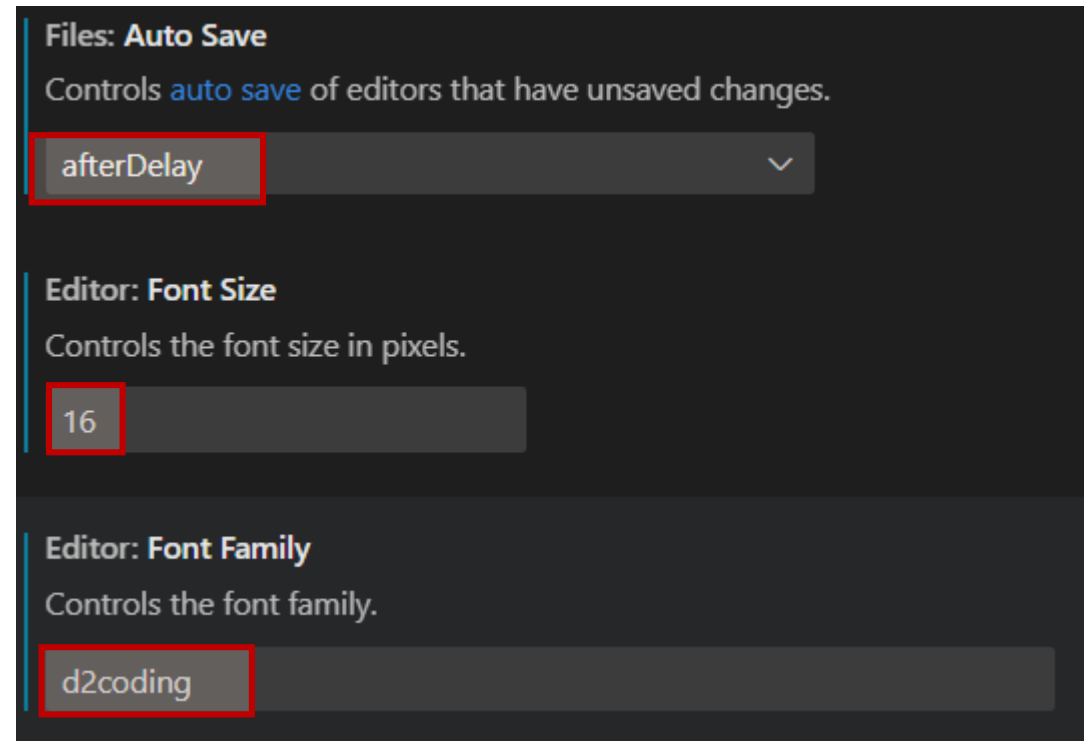
# 개발 환경 설정하기

- (실습) 비주얼 스튜디오 코드 설치하기

## 2. VS Code 환경 설정과 도구 설치



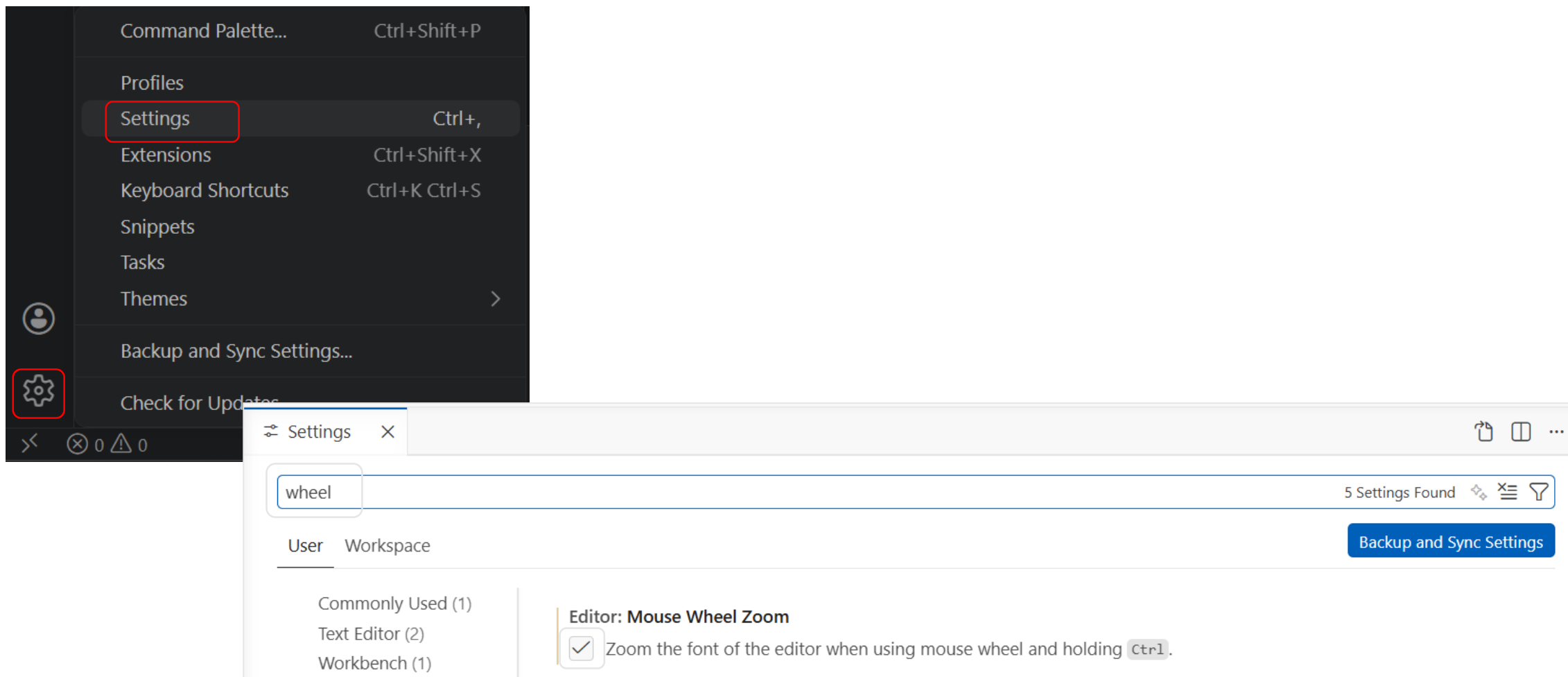
- 자동 저장 기능 설정(Auto Save/Auto Save Delay:1000)
- 폰트 사이즈 설정(Font Size)
- 글꼴 설정(Font Family)



# 개발 환경 설정하기

- (실습) 비주얼 스튜디오 코드 설치하기

## 2. VS Code 환경 설정과 도구 설치

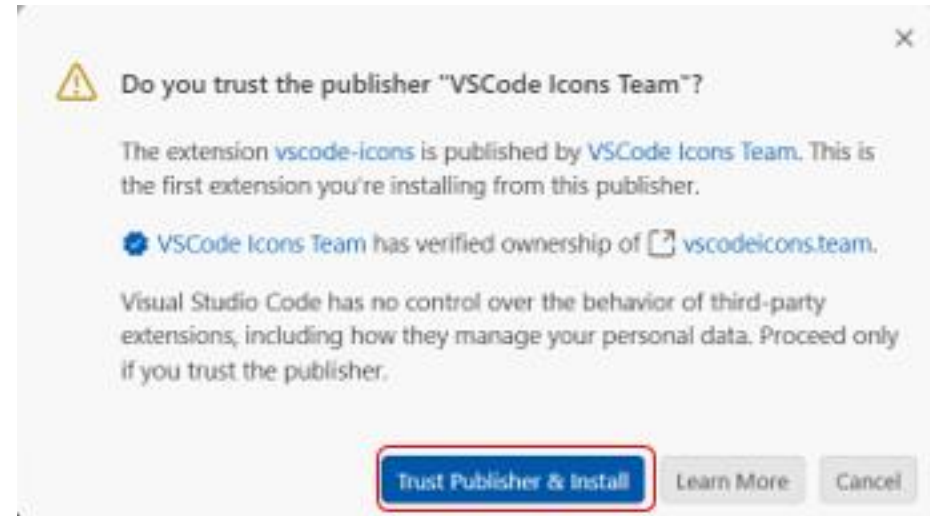
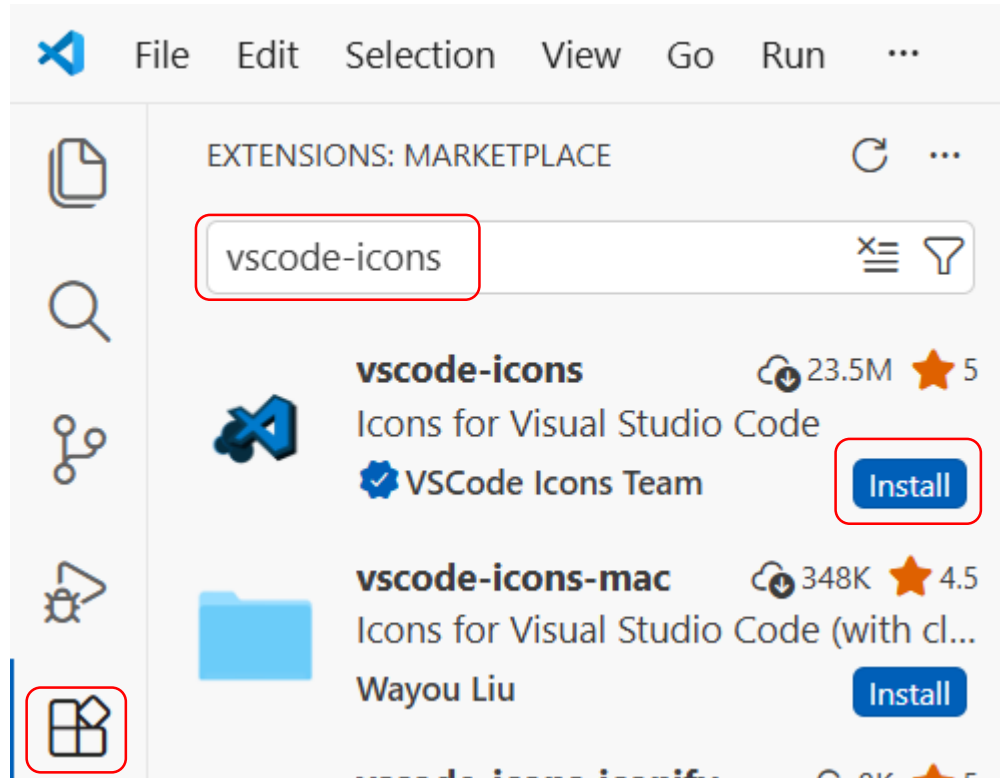




# 개발 환경 설정하기

- (실습) 비주얼 스튜디오 코드 설치하기

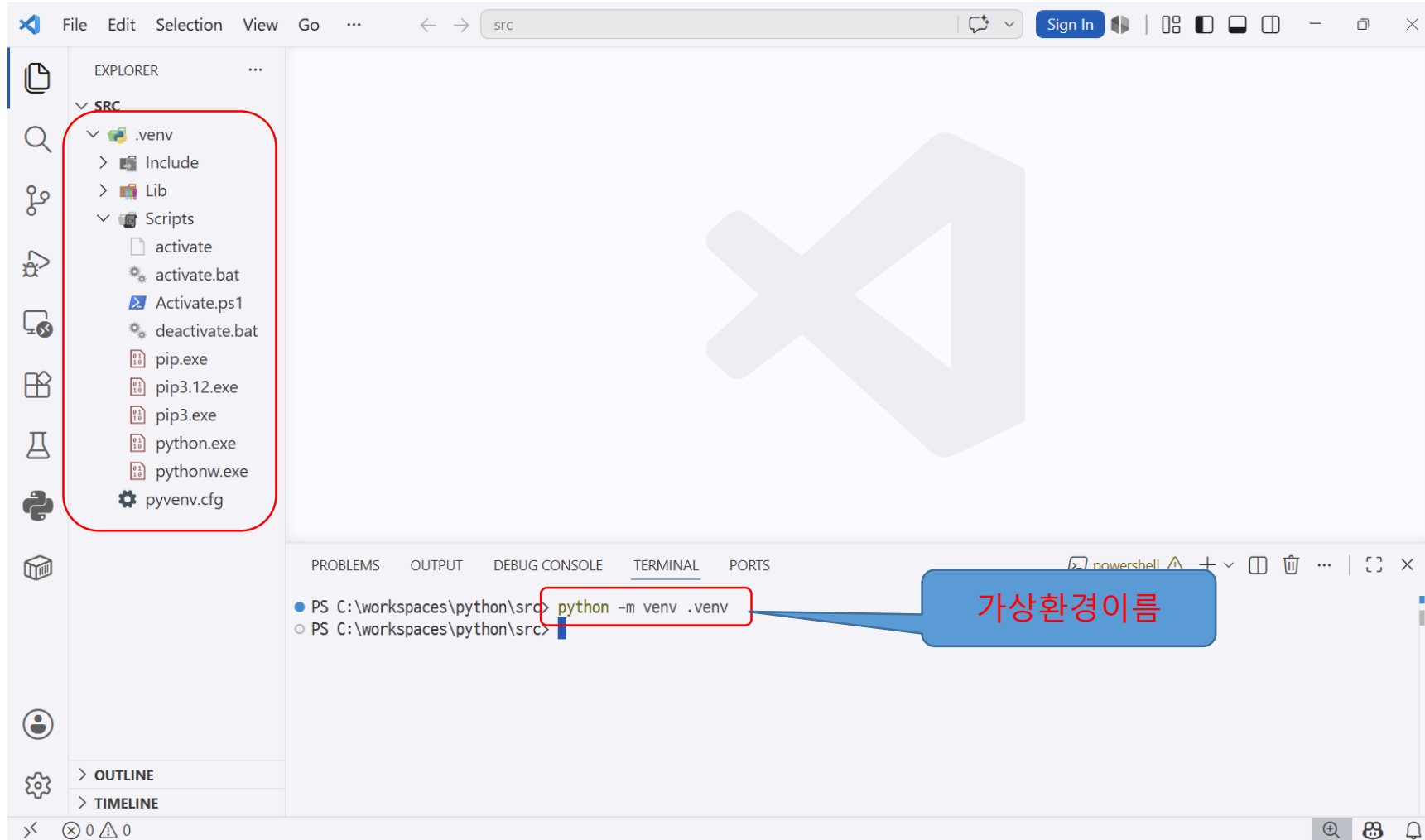
3. vscode-icons 확장 도구 설치



# 개발 환경 설정하기

- (실습) 가상 환경 만들기

1. VS Code에서 프로젝트 폴더로 사용할 폴더를 열고 **Ctrl** + **'** 을 눌러 터미널 창 열기



# 개발 환경 설정하기

- (실습) 가상 환경 만들기

- 2. VS Code의 터미널 창에 다음과 같이 입력해 가상 환경 활성화



The screenshot shows the VS Code interface with the 'TERMINAL' tab selected. The terminal is running PowerShell. The first command entered is `python -m venv .venv`. The second command, `.\.venv\Scripts\activate`, is highlighted with a red box. The output shows the prompt changing from `PS C:\workspaces\python\src>` to `(.venv) PS C:\workspaces\python\src>`, with `(.venv)` also highlighted by a red box.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell ! + v [ ] [ ] ... | [ ] X
```

```
● PS C:\workspaces\python\src> python -m venv .venv
```

```
● PS C:\workspaces\python\src> .\.venv\Scripts\activate
```

```
○ (.venv) PS C:\workspaces\python\src>
```

# 가상 환경 생성

```
PS C:\workspaces\python\src> python -m venv .venv
```

# 윈도우 가상 환경 활성화

```
PS C:\workspaces\python\src> .\venv\Scripts\activate # 맥 환경에서는 Scripts를 bin으로 변경
```

```
(.venv) PS C:\workspaces\python\src>
```

# 개발 환경 설정하기

- (실습) 가상 환경 만들기

- 가상 환경을 만들 때 오류가 발생한다면?
  - 오류 메시지

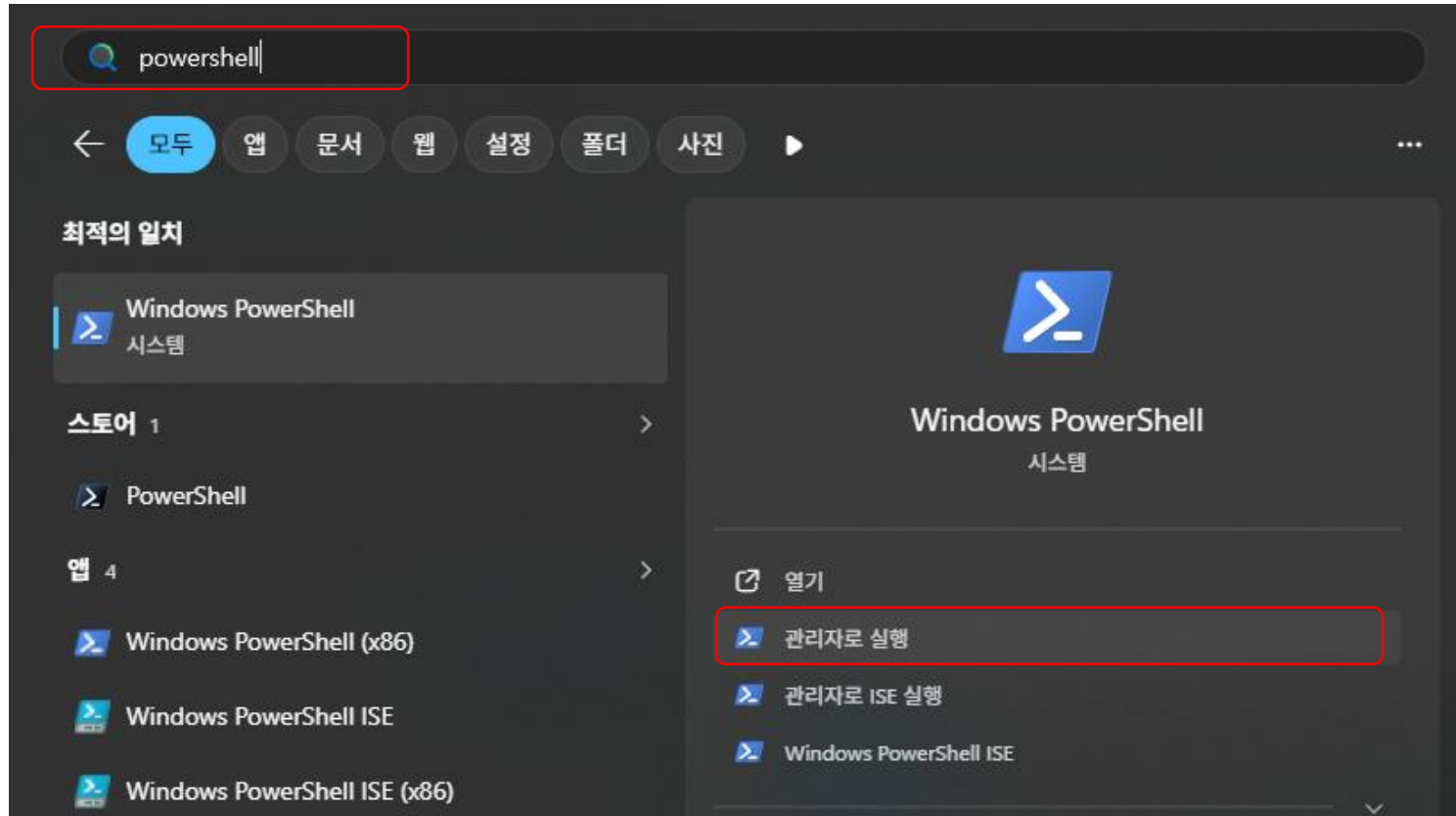
```
PS C:\workspaces\ai_agent> python -m venv .venv
PS C:\workspaces\ai_agent> .\ .venv \Scripts\activate
.\ .venv \Scripts\activate : 이 시스템에서 스크립트를 실행할 수 없으므로 C:\workspaces\ai_agent\.venv\Scripts\Activate.ps1 파일을 로드할 수 없습니다. 자세한 내용은 about_Execution_Policies(https://go.microsoft.com/fwlink/?LinkID=135170)를 참조하십시오.
위치 줄:1 문자:1
+ .\ .venv \Scripts\activate
+ ~~~~~
+ CategoryInfo          : 보안 오류: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\workspaces\ai_agent>
```

# 개발 환경 설정하기

- (실습) 가상 환경 만들기

- 가상 환경을 만들 때 오류가 발생한다면?

1. 윈도우 검색 창에 'powershell'을 검색하고 [관리자로 실행] 클릭



# 개발 환경 설정하기

---

- (실습) 가상 환경 만들기

- 가상 환경을 만들 때 오류가 발생한다면?

2. get-ExecutionPolicy를 입력해 현재 실행 정책 확인

윈도우 파워셸

```
PS C:\WINDOWS\system32> get-ExecutionPolicy
```

# 개발 환경 설정하기

- (실습) 가상 환경 만들기

- 가상 환경을 만들 때 오류가 발생한다면?

3. 실행 정책을 변경하기 위해 **Set-ExecutionPolicy RemoteSigned**를 입력하고 이어서 y를 입력

스크립트 실행이 허용되지 않는 상태

```
원도우 파워셸

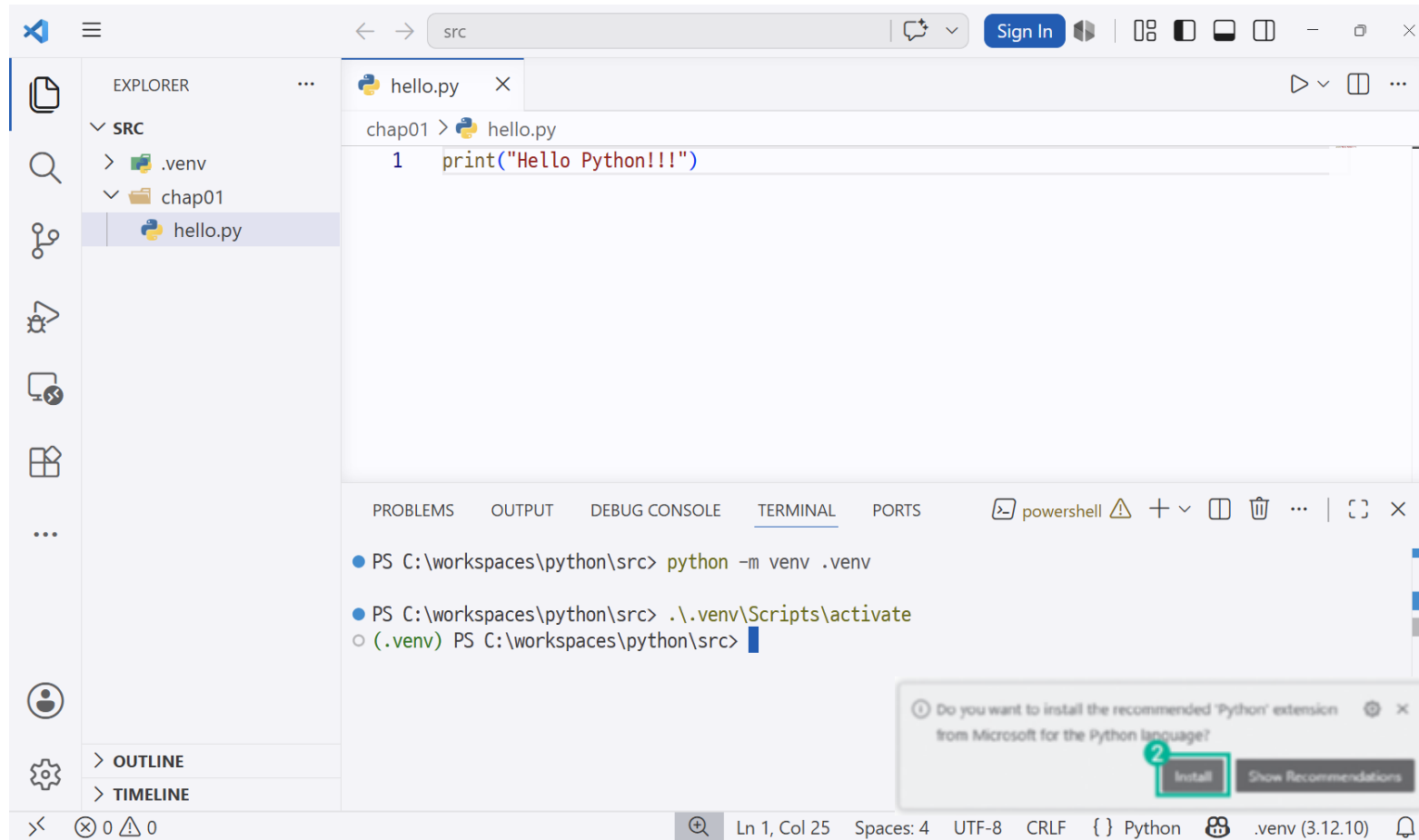
PS C:\WINDOWS\system32> get-ExecutionPolicy
Restricted
PS C:\WINDOWS\system32> Set-ExecutionPolicy RemoteSigned
실행 규칙 변경
실행 정책은 신뢰하지 않는 스크립트로부터 사용자를 보호합니다. 실행 정책을 변경하면 about_Execution_Policies 도움말
항목(https://go.microsoft.com/fwlink/?LinkID=135170)에 설명된 보안 위험에 노출될 수 있습니다. 실행 정책을
변경하시겠습니까?
[Y] 예(Y) [A] 모두 예(A) [N] 아니요(N) [L] 모두 아니요(L) [S] 일시 중단(S) [?] 도움말
(기본값은 "N"):
```

# 개발 환경 설정하기

- (실습) 비주얼 스튜디오 코드 설치하기

- 3. VS Code를 실행하고 확장자명이 .py인 파이썬 파일 생성

- VS Code의 파이썬 익스텐션 설치





# 개발 환경 설정하기

- (실습) 가상 환경 만들기

4. VS Code의 터미널 창에 다음과 같이 입력해 **파이썬 실행**
5. VS Code의 터미널 창에 다음과 같이 입력해 **가상 환경 비활성화**
  - 비활성화 되는 것을 확인했으면, 반드시 다시 가상 환경으로 활성화 해둘 것



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  powershell  + -  [ ]  [X]  ...  [ ]  X
```

- PS C:\workspaces\python\src> python -m venv .venv
- PS C:\workspaces\python\src> .\.venv\Scripts\activate
- (.venv) PS C:\workspaces\python\src> python .\chap01\hello.py # 파이썬 실행  
Hello Python!!!
- (.venv) PS C:\workspaces\python\src> deactivate # 가상환경 비활성화
- PS C:\workspaces\python\src>

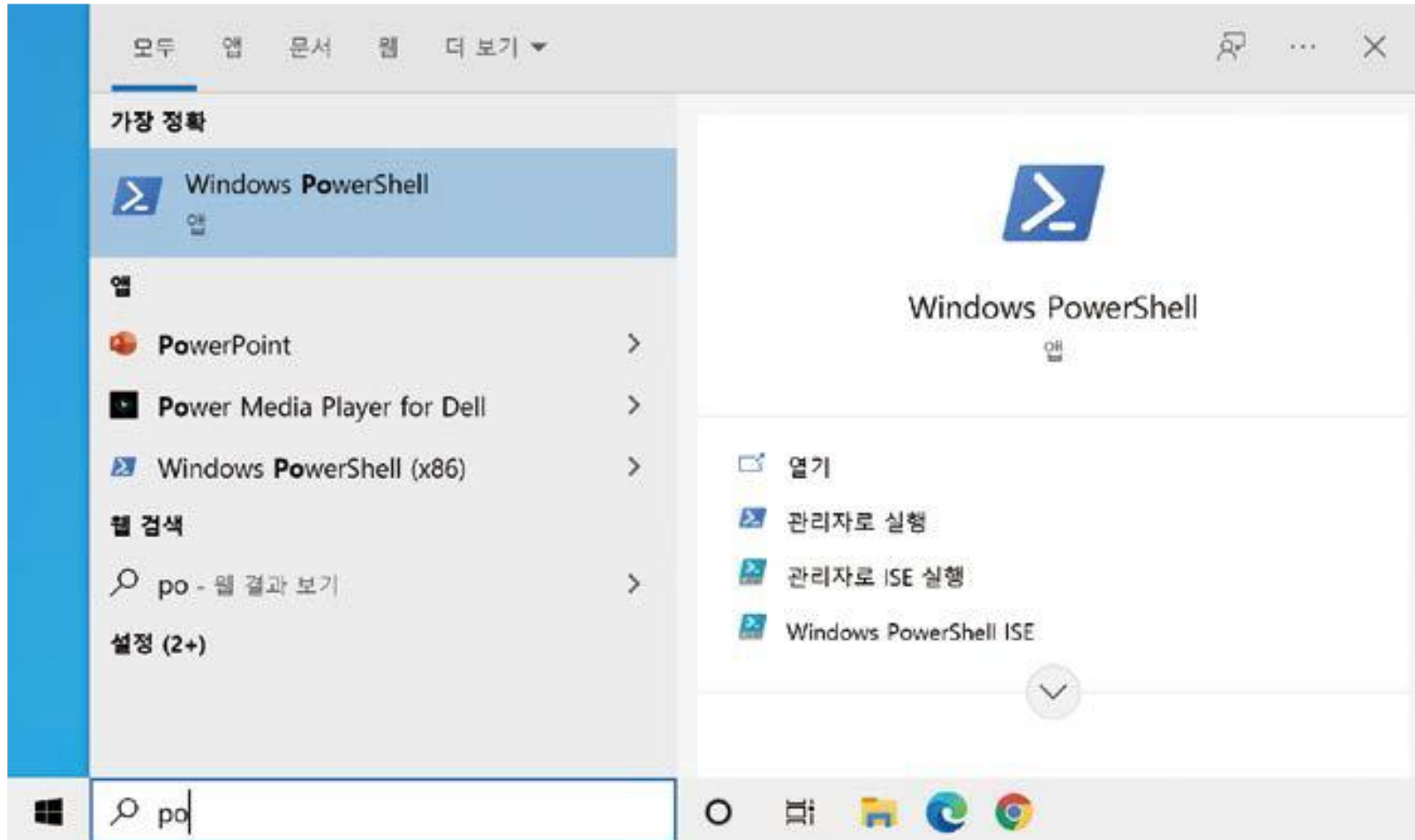
# (참조) 코드 실행기 사용하기: 윈도우 파워셸

---

- 파이썬 인터랙티브 셸을 활용해서 파이썬 코드를 작성하고 실행하는 것보다는
  - 1) 비주얼 스튜디오 코드로 코드를 작성
  - 2) 파워셸 또는 터미널 등의 셸에서 파이썬 명령어를 입력해서 코드를 실행하는 것을 추천
- 파이썬 기초 단계를 넘어가면 더 이상 IDLE를 사용하지 않지만, 프롬프트에서 명령어를 입력해서 실행하는 방법은 계속 활용하기 때문

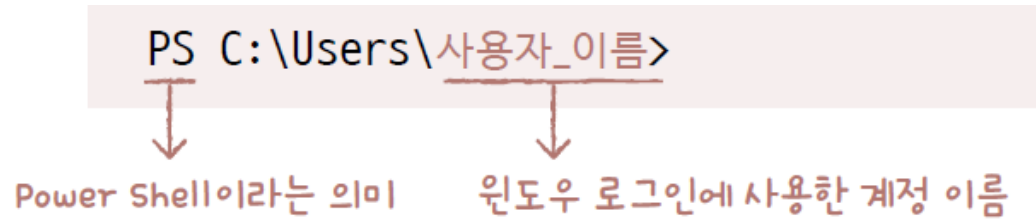
# (참조) 코드 실행기 사용하기: 윈도우 파워셸

- [Windows PowerShell] 실행



# (참조) 코드 실행기 사용하기: 윈도우 파워셸

- 현재 폴더는 'C:\Users\사용자\_이름'에 위치



PS C:\Users\사용자\_이름>

↓                      ↓

Power Shell이라는 의미    윈도우 로그인에 사용한 계정 이름

The diagram shows a Windows PowerShell command prompt window with the text 'PS C:\Users\사용자\_이름>'. Below this, two red arrows point downwards. The first arrow points from 'PS' to the text 'Power Shell이라는 의미'. The second arrow points from '사용자\_이름' to the text '윈도우 로그인에 사용한 계정 이름'.

# (참조) 코드 실행기 사용하기: 윈도우 파워셸

- 현재 폴더 확인하기: ls 명령어

- ls 명령어 또는 dir 명령어, Get-ChildItem 명령어를 사용

```
PS C:\Users\사용자_이름> ls
```

```
디렉터리: C:\Users\사용자_이름
```

- ls 명령어 실행 결과가 너무 많이 출력될 때, explorer 명령어로 탐색기를 실행. 마침표(.)를 사용하면 익숙한 폴더 창 형태로 현재 폴더 내용을 확인

```
PS C:\Users\사용자_이름> explorer . → 명령어 뒤에 마침표를 한 칸 띄어씁니다.
```

```
PS C:\Users\사용자_이름>
```

# (참조) 코드 실행기 사용하기: 윈도우 파워셸

- 폴더로 이동하기: cd 명령어

- (예) 바탕화면 (Desktop) 폴더로 이동

```
PS C:\Users\사용자_이름> cd Desktop
```

```
PS C:\Users\사용자_이름\Desktop> → 실행 위치가 Desktop 폴더로 변경됩니다.
```

- 다시 상위 폴더로 돌아가려면 cd .. 명령어를 사용

```
PS C:\Users\사용자_이름\Desktop> cd .. → 마침표 2개(..)는 상위 폴더를 의미합니다.
```

```
PS C:\Users\사용자_이름>
```

- 폴더 이름에 띄어쓰기가 포함되어 있을 때는 큰따옴표나 작은따옴표를 사용

```
PS C:\Users\사용자_이름> cd "파이썬 예제"
```

```
PS C:\Users\사용자_이름\파이썬 예제>
```

# (참조) 코드 실행기 사용하기: 윈도우 파워셸

- 파이썬 파일 실행하기: python 명령어

- 명령어를 실행하는 위치와 파이썬 파일이 있는 위치는 일치해야 함

(예) 바탕 화면 'python\_sample'라는 폴더의 'test.py'라는 파이썬 파일을 실행

```
PS C:\Users\사용자_이름> cd Desktop →바탕화면으로 이동합니다.
```

```
PS C:\Users\사용자_이름\Desktop> cd python_sample →python_sample 폴더로 이동합니다.
```

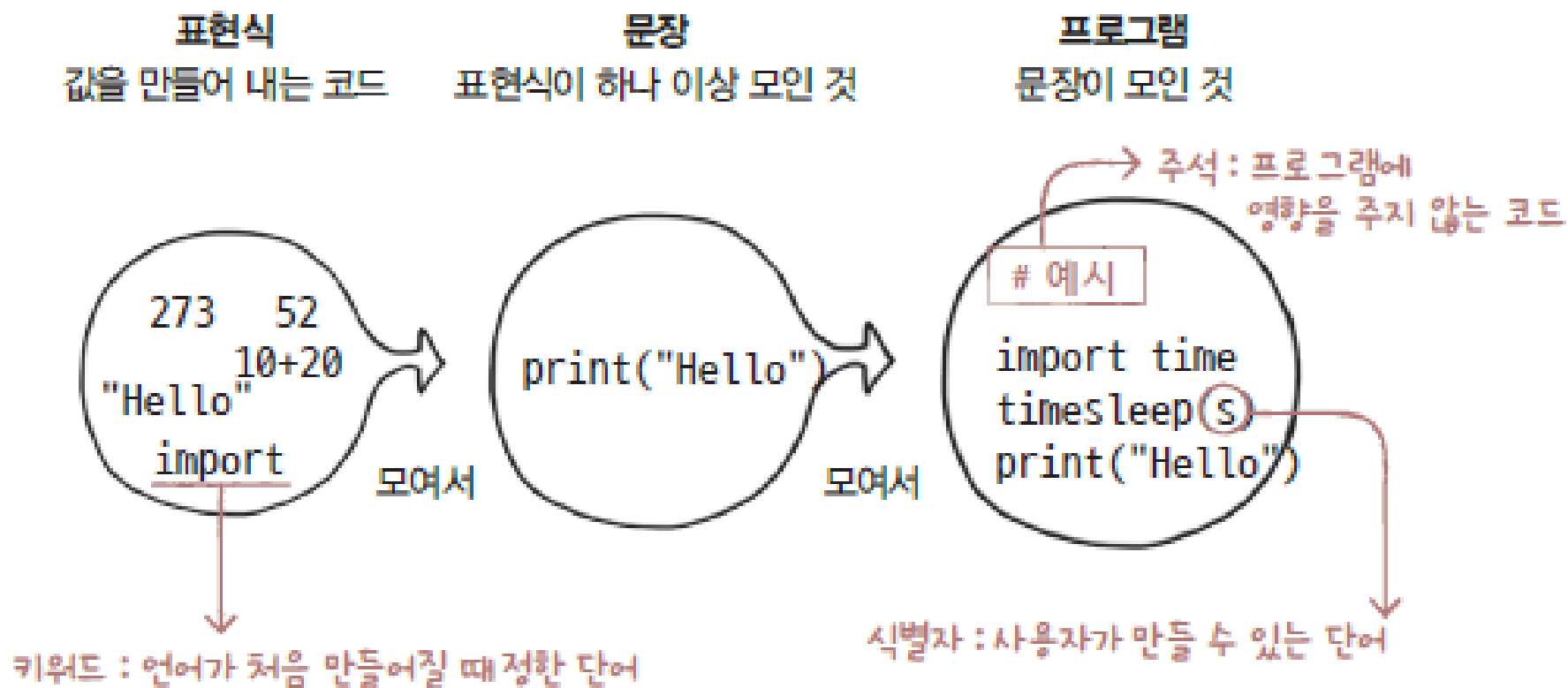
```
PS C:\Users\사용자_이름\Desktop\python_sample> python test.py →파이썬 파일을 실행합니다.
```

- cd 명령어를 여러 번 사용하여 폴더 경로를 입력하여 한번에 실행

```
PS C:\Users\사용자_이름> cd c:\Users\사용자_이름\Desktop\python_sample
```

```
PS C:\Users\사용자_이름\Desktop\python_sample> python test.py
```

# 파이썬 용어들





# 문장과 표현식

---

- 문장 (statement)

- 실행할 수 있는 코드의 최소 단위
- 파이썬은 '한 줄이 하나의 문장이다'

```
# 실행되는 모든 한 줄 코드는 문장입니다.  
print("Python Programming") # 문장  
10 + 20                      # 문장
```

- 프로그램 (program)

- 문장이 모여서 형성

# 문장과 표현식

---

- 표현식 (expression)
  - 파이썬에서 어떠한 값을 만들어내는 간단한 코드
  - 값이란 숫자 수식, 문자열 등이 될 수 있음

```
273
```

```
10 + 20 + 30 * 10
```

```
"Python Programming"
```

# 키워드

- 키워드 (keyword)

- 특별한 의미가 부여된 단어
- 파이썬에서 이미 특정 의미로 사용하기로 예약해 놓은 것
- 프로그래밍 언어에서 이름 정할 때 똑같이 사용할 수 없음

|       |        |          |       |        |          |
|-------|--------|----------|-------|--------|----------|
| False | None   | True     | and   | as     | assert   |
| break | class  | continue | def   | del    | elif     |
| else  | except | finally  | for   | from   | global   |
| if    | import | in       | is    | lambda | nonlocal |
| not   | or     | pass     | raise | return | try      |
| while | with   | yield    |       |        |          |

- 대소문자 구별

# 키워드

---

- 아래 코드로 특정 단어가 파이썬 키워드인지 확인 가능

```
>>> import keyword  
>>> print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',  
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',  
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',  
'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

# 식별자

## ● 식별자 (identifier)

- 프로그래밍 언어에서 이름 붙일 때 사용하는 단어
- 변수 또는 함수 이름 등으로 사용
- 키워드 사용 불가
- 특수문자는 언더바(\_)만 허용
- 숫자로 시작 불가
- 공백 포함 불가
- 알파벳 사용이 관례
- 의미 있는 단어로 할 것

| 사용 가능한 단어 | 사용 불가능한 단어                  |
|-----------|-----------------------------|
| alpha     | break → 키워드라서 안 됩니다.        |
| alpha10   |                             |
| _alpha    | 273alpha → 숫자로 시작해서 안 됩니다.  |
| AlpHa     |                             |
| ALPHA     | has space → 공백을 포함해서 안 됩니다. |

# 식별자

- 스네이크 케이스와 캐멀 케이스

`itemlist`      `loginstatus`      `characterhp`      `rotateangle`

- 공백이 없어 이해하기 어려움
  - 스네이크 케이스 (snake case) : 언더바(\_)를 기호 중간에 붙이기
  - 캐멀 케이스 (camel case) : 단어들의 첫 글자를 대문자로 만들기

| 식별자에 공백이 없는 경우                                                                                            | 단어 사이에 _ 기호를 붙인 경우<br>(스네이크 케이스)                                                                              | 단어 첫 글자를 대문자로 만든 경우<br>(캐멀 케이스)                                                                           |
|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>itemlist</code><br><code>loginstatus</code><br><code>characterhp</code><br><code>rotateangle</code> | <code>item_list</code><br><code>login_status</code><br><code>character_hp</code><br><code>rotate_angle</code> | <code>ItemList</code><br><code>LoginStatus</code><br><code>CharacterHp</code><br><code>RotateAngle</code> |

- 파이썬에서는 스네이크 및 캐멀 케이스 둘 모두 사용

# 식별자

- 식별자 구분하기

- 캐멀 케이스에서는 첫 번째 글자를 소문자로 적지 않음

캐멀 케이스 유형 1 : PrintHello → 파이썬에서 사용됩니다.

캐멀 케이스 유형 2 : printHello → 파이썬에서 사용하지 않습니다.

print

input

list

str

map

filter

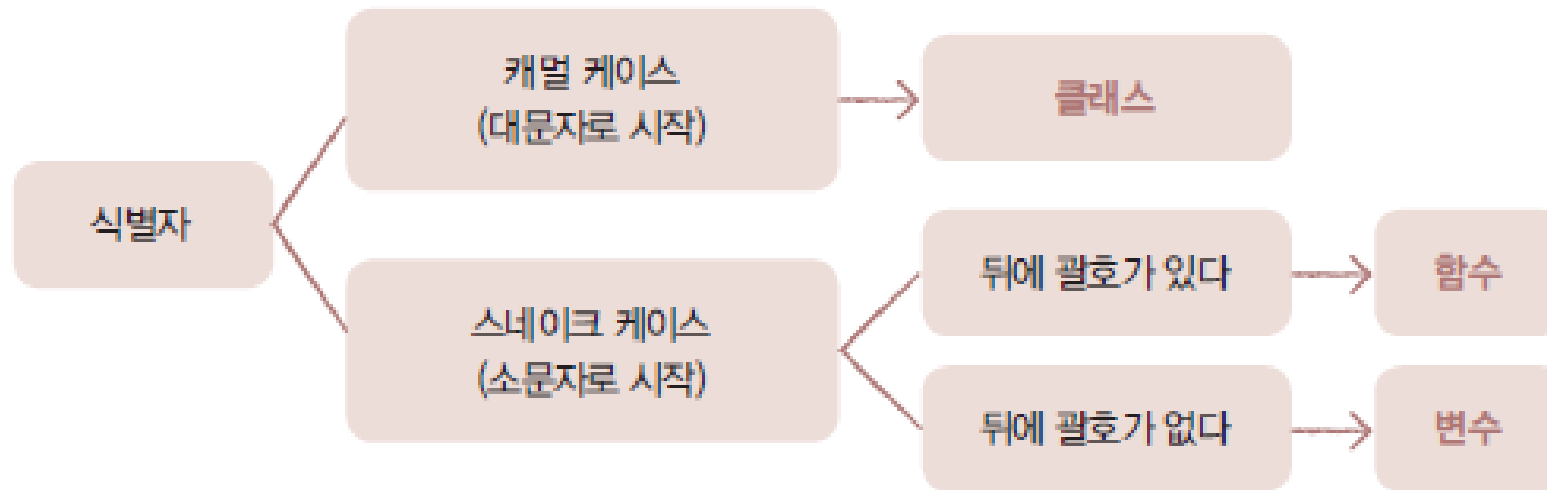
- 첫 번째 글자가 대문자라면 무조건 캐멀 케이스

Animal

Customer

# 식별자

- 캐멀 케이스로 작성되었으면 클래스
- 스네이크 케이스로 작성되어 있으면 함수 또는 변수
- 뒤에 괄호 붙으면 함수
- 뒤에 괄호 없으면 변수





# 식별자

---

- 식별자가 클래스인지, 변수인지, 함수인지 구분해 봅시다

```
1. print()
2. list()
3. soup.select()
4. math.pi
5. math.e
6. class Animal:
7. BeautifulSoup()
```

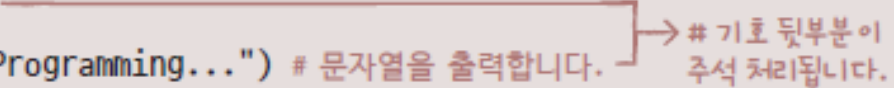
# 주석

---

- 주석 (comment)

- 프로그램 진행에 영향 주지 않는 코드
- 프로그램 설명 위해 사용
- # 기호를 주석으로 처리하고자 하는 부분 앞에 붙임

```
>>> # 간단히 출력하는 예입니다.  
>>> print("Hello! Python Programming...") # 문자열을 출력합니다.  
Hello! Python Programming...
```



# 기호 뒷부분이  
주석 처리됩니다.

# 연산자와 자료

---

- 연산자

- 스스로 값이 되는 것이 아닌 값과 값 사이에 무언가 기능 적용할 때 사용

```
>>> 1 + 1
2
>>> 10 - 10
0
```

- 리터럴 (literal)

- 자료 = 어떠한 값 자체

```
1
10
"Hello"
```

# 출력 : print()

---

- print() 함수

- 출력 기능
- 출력하고 싶은 것들을 괄호 안에 나열

```
print(출력1, 출력2, ...)
```

- 하나만 출력하기

```
>>> print("Hello! Python Programming...")
Hello! Python Programming...
>>> print(52)
52
>>> print(273)
273
```

# 출력 : print()

---

- 여러 개 출력하기

```
>>> print(52, 273, "Hello")
52 273 Hello
>>> print("안녕하세요", "저의", "이름은", "윤인성입니다!")
안녕하세요 저의 이름은 윤인성입니다!
```

- 줄바꿈하기

```
>>> print()
      → 빈 줄을 출력합니다.
>>>
```

# 출력 : print()

---

- 예시 – 기본 출력

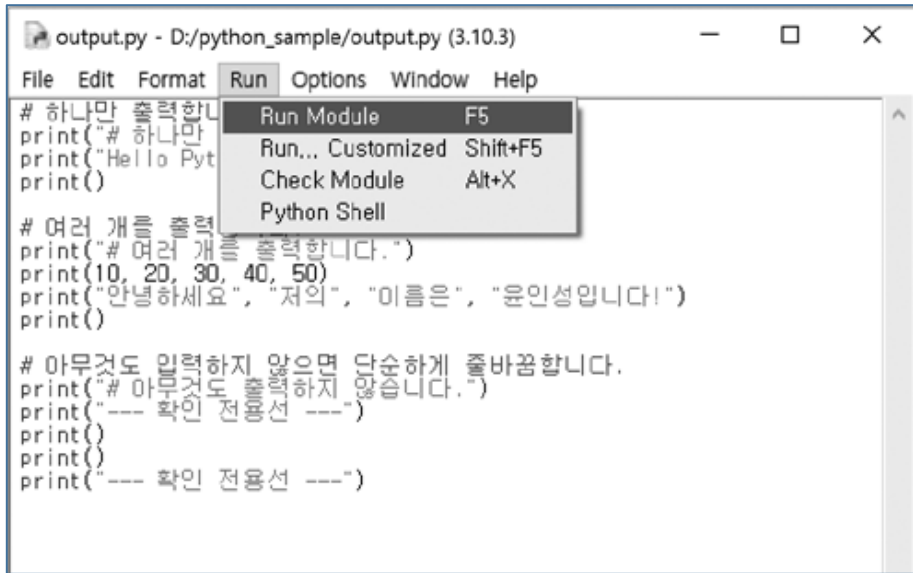
---

```
01  # 하나만 출력합니다.
02  print("# 하나만 출력합니다.")
03  print("Hello Python Programming...!")
04  print()
05
06  # 여러 개를 출력합니다.
07  print("# 여러 개를 출력합니다.")
08  print(10, 20, 30, 40, 50)
09  print("안녕하세요", "저의", "이름은", "윤인성입니다!")
10  print()
11
12  # 아무것도 입력하지 않으면 단순히 줄바꿈합니다.
13  print("# 아무것도 출력하지 않습니다.")
14  print("— 확인 전용선 —")
15  print()
16  print()
17  print("— 확인 전용선 —")
```

---

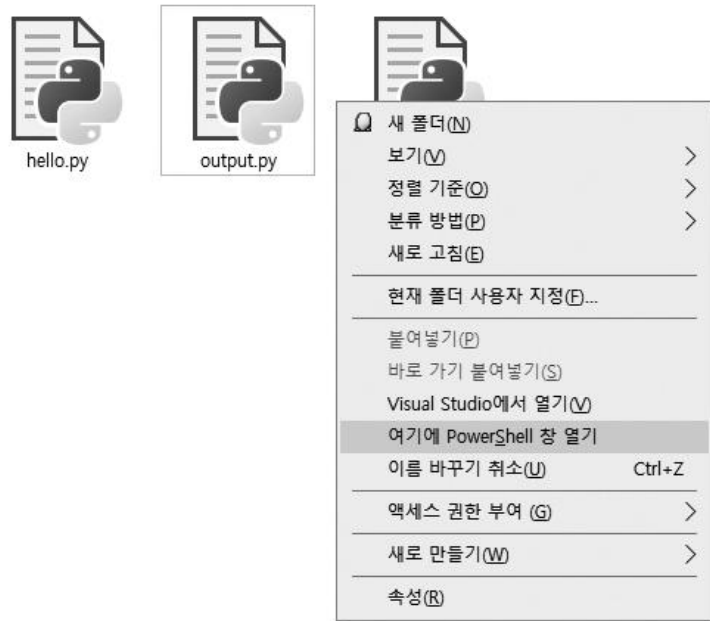
# 출력 : print()

- 파이썬 IDLE 에디터에서의 실행
  - [Run] – [Run Module] 선택



# 출력 : print()

- 비주얼 스튜디오 코드에서의 실행
  - 탐색기에서 파일 저장한 폴더로 이동하여 Shift 누른 상태로 마우스 우클릭
  - [여기에 PowerShell 창 열기]





# 출력 : print()

---

- 명령 프롬프트 실행되면 python 명령어 사용하여 해당 파일 실행

```
> python output.py
```

```
# 하나만 출력합니다.
```

```
Hello Python Programming...!
```

```
# 여러 개를 출력합니다.
```

```
10 20 30 40 50
```

```
안녕하세요 저의 이름은 윤인성입니다!
```

```
# 아무것도 출력하지 않습니다.
```

```
--- 확인 전용선 ---
```

```
--- 확인 전용선 ---
```